RA26-Challenge 6: Enunciado Entrenamiento

DMA, scroll, mapas, window, niveles, IA, coordenadas y animación

CHELDA ⚔️

Un mini-Zelda visto desde arriba. Caminas por un mundo grande, esquivas enemigos, los derrotas con la espada y buscas el portal que te lleva al siguiente nivel. Vas a programar y a corregir **10 piezas** del juego: la copia de sprites con DMA, la cámara, las paredes del mapa, el HUD, la carga de niveles, la IA de los enemigos, el cambio de coordenadas y la animación.

Todas son rutinas del juego. Juntas forman un juego completo: cuando termines, el juego funciona con tu código.

> 🔥 **Este documento solo tiene los ejercicios**
> Todo lo que necesitas saber para resolverlos está en la guía de aprendizaje. Cada ejercicio indica su **bloque**. Lee el bloque y, después, resuelve el ejercicio.

> ⓘ **Trabaja como en clase**
> 1. **En papel.** Piensa, dale vueltas, intenta, falla, prueba de nuevo.
> 2. **En GBTelera**. Copia el código, ejecuta, depura, haz funcionar el juego.

Entrena para el challenge

Tenéis **10 ejercicios**. En clase serán estos mismos, con adaptaciones, y alguno vendrá partido en dos: **12 en total**. El juego y el proyecto no cambiarán.

Lo importante es que sepas desarrollarlos por ti mismx. Crea de cero tu propia solución y comprende lo que haces: así sabrás adaptarla sin problema.

> ⓘ **Dos formatos de ejercicio**
> - **Implementar o completar**: rutina vacía o con parte del código escrito. Escribe lo que falta en `◉◉YOUR CODE◉◉`.
> - **Corregir**: el código viene **con errores de todo tipo**. Encuéntralos y arréglalos primero en papel, después en el proyecto (`ej01.asm.no` y `ej10.asm.no`). Renombra cada fichero a `.asm` y empieza.

> ⓘ **Tiempo orientativo**
> Es el tiempo de **resolver** cada ejercicio entero: entenderlo, hacerlo en papel, pasarlo al ordenador y depurarlo. Leer el bloque de aprendizaje correspondiente son unos 15-20 minutos más.

Tiempo orientativo

Ej.	Bloque	Rutinas	Minutos
1	1	`DMARoutine`, `dma_init` (corregir)	~30
2	2	`camera_axis`, `sys_render_scroll`	~40
3	3	`map_tile_address`, `map_is_solid_at`	~55
4	4	`hud_show`, `hud_draw_hearts`	~40
5	5	`game_enemies_spawn`	~45
6	6	`sys_ai_one`	~40
7	6	`sys_ai_choose`	~30
8	7	`world_to_oam`	~30
9	7	`sys_prerender_one`	~45
10	8	`sys_animation_one` (corregir)	~65
		Total	**~7 h**

Para equipos de 5 personas (2 ejercicios cada una): **unas 2 h por persona**, de ellas 1,5 h de ejercicios y 0,5 h de lectura de sus bloques.
Entrenamiento completo (todos los ejercicios y los 8 bloques): unas 9 h. Mejor 1 h al día como máximo.
Los ejercicios 6 y 7 (IA) y 8 y 9 (coordenadas) son del mismo tema: el segundo de cada pareja practica otra parte de la misma idea.

Organización del proyecto

Carpetas por función: `engine/` (servicios), `assets/` (datos), `man/` (gestores), `sys/` (sistemas) y `scn/` (escenas).

```
src/
├── main.asm                Bucle principal, y líneas DEF que eliges (TEST, EJ01…EJ10)
├── engine/                 Servicios: pantalla, memoria, DMA, texto, joypad, ...
│   ├── dma.asm             Ej. 1
│   ├── gfx/ snd/ lib/      Gráficos, sonido y librerías (dados)
│   └── ui/hud.asm          Ej. 4
├── assets/                 Los datos del juego (dados): gfx, snd, levels
├── man/
│   └── man_map.asm         Ej. 3
├── sys/                    Sistemas: un fichero cada uno
│   ├── sys_camera.asm      Ej. 2       sys_render.asm       Ej. 2
│   ├── sys_ai.asm          Ej. 6, 7    sys_prerender.asm    Ej. 8, 9
│   └── sys_animation.asm   Ej. 10
├── scn/                    Escenas
│   └── scn_game_rules.asm  Ej. 5 (y reglas dadas)
├── chcode/                 Pruebas y código del proyecto (no tocar)
└── ej01.asm … ej10.asm     ← TU CÓDIGO: un fichero por ejercicio (en el proyecto)
    (los de corregir, 1 y 10, empiezan como ej01.asm.no y ej10.asm.no, con errores)
```

ⓘ Cómo se integra tu código
Ejemplo:

sys/sys_ai.asm

```
sys_ai_one::
    jp ej06   ;; Salto a tu código
```

Tú implementas `ej06::` en `ej06.asm` y tu código se ejecuta cuando el juego llame a `sys_ai_one::`.

Comprueba cada ejercicio

Cada ejercicio tiene dos casos de prueba, **P1 y P2** (como en el Challenge 3).

1. En `main.asm`: `DEF TEST EQUS "test_ej05"`. Ejecuta `make` y abre `game.gb` en BGB.
2. BGB se para en cada `ld b, b`. Compara CPU y MEM con el RESULTADO ESPERADO del final de `ej05.asm`.
3. Las pruebas usan las rutinas del proyecto para todo lo demás: un fallo en otro ejercicio no afecta al tuyo.

Para probarlo dentro del juego: `DEF TEST EQUS "game"` y `DEF EJ05 = 1`. Con `0`, el juego usa la rutina del proyecto. Puedes poner a `1` todos los que quieras.

En clase no habrá rutinas del proyecto para el juego: funcionará cuando estén todos los ejercicios. Si funciona, **bonus**.

CHALLENGE

Código común

⚠ **Este código es común para todos los ejercicios. Tus soluciones deben usarlo.**

man_entity.inc: la entidad, los componentes y las direcciones

```
;;=============================================================
;; Challenge 6 · Gestor de entidades: estructura y componentes
;; ❌ No lo modifiques
;;=============================================================
IF !DEF(MAN_ENTITY_INC)
DEF MAN_ENTITY_INC = 1
;;-------------------------------------------------------------
;; Entities: array of MAX_ENTITIES entities of SIZEOF_ENTITY bytes
;;-------------------------------------------------------------
DEF MAX_ENTITIES = 12
RSRESET
DEF E_CMP         RB 1          ;; 0  Which components the entity has (bitflags)
DEF E_SPRITE      RB 1          ;; SPRITE (offset): the same 4 bytes as an OAM entry
DEF E_Y =     E_SPRITE          ;; 1  ┐ E_Y = E_SPRITE. WORLD coordinates in pixels
DEF E_X           RB 1          ;; 2  │ (top left of its image): 0-255, as the map
DEF E_TILE        RB 1          ;; 3  │ TILE: the first tile of the image it shows now
DEF E_ATTR        RB 1          ;; 4  ┘ ATTR: flips (and palette) of its sprites
DEF E_VY          RB 1          ;; 5  ┐ PHYSICS: velocity (signed) in pixels per frame
DEF E_VX          RB 1          ;; 6  ┘
DEF E_KIND        RB 1          ;; 7  COLLIDER: what kind of thing this entity is
DEF E_DIR         RB 1          ;; 8  The way it faces: DIR_DOWN, DIR_UP... (below)
DEF E_SHEET       RB 1          ;; 9  ANIMATION: the first tile of its sheet of images
DEF E_TICK        RB 1          ;; 10 ANIMATION: frames it has been walking (its step)
DEF E_LIFE        RB 1          ;; 11 SELFDESTROY: frames left before it disappears
DEF E_THINK       RB 1          ;; 12 AI: frames left until it chooses where to go
DEF E_RESERVED    RB 3          ;; 13 Free: for the components of your own project
DEF SIZEOF_ENTITY RB 0          ;; = 16
DEF ENTITY_ARRAY_SIZE = MAX_ENTITIES * SIZEOF_ENTITY

;;-------------------------------------------------------------
;; ALIGNMENT: what the entity manager guarantees to every system
;; The array is in ONE page: it starts at an address ending in $00 (ALIGN[8]) and
;; it is not larger than 256 bytes. Therefore:
;;  1. The HIGH byte is the same for ALL the entities and for all their fields.
;;     Going to another entity or to another field only changes the LOW byte
;;     (so "add hl, de" is not needed).
;;  2. Adding an offset to the low byte never carries: the address of a field is
;;     the low byte of the entity + the offset of the field. Walking from a field
;;     to the next one is "inc l".
;;  3. An entity can be remembered with ONE byte: its low byte. ENTITY_NONE is not
;;     an entity (every entity starts below the size of the array), so it means
;;     "there is none".
;; The code relies on this, so it is checked when the project is built:
;;-------------------------------------------------------------
DEF ENTITY_NONE = $FF
```

```
ASSERT ENTITY_ARRAY_SIZE <= ENTITY_NONE

;; The image of an entity: 16 x 16 pixels (2 sprites of 8 x 16, side by side)
DEF IMAGE_WIDTH  = 16
DEF IMAGE_HEIGHT = 16

;; Bits of CMP. CMP = 0 → the entity is FREE.
;; Changing the bits while the game runs changes what the entity does
DEF CMP_SPRITE_BIT      = 0
DEF CMP_PHYSICS_BIT     = 1
DEF CMP_COLLIDER_BIT    = 2
DEF CMP_SELFDESTROY_BIT = 3
DEF CMP_INPUT_BIT       = 4
DEF CMP_ANIMATION_BIT   = 5
DEF CMP_AI_BIT          = 6
DEF CMP_ALIVE_BIT       = 7
;; Bit Masks for CMPs
DEF CMP_SPRITE          = 1 << CMP_SPRITE_BIT      ;; It is drawn (2 sprites of 8x16)
DEF CMP_PHYSICS         = 1 << CMP_PHYSICS_BIT     ;; It moves, and the map stops it
DEF CMP_COLLIDER        = 1 << CMP_COLLIDER_BIT    ;; It collides with other entities
DEF CMP_SELFDESTROY     = 1 << CMP_SELFDESTROY_BIT ;; It disappears after E_LIFE frames
DEF CMP_INPUT           = 1 << CMP_INPUT_BIT       ;; Controlled by the player
DEF CMP_ANIMATION       = 1 << CMP_ANIMATION_BIT   ;; Its image follows step and DIR
DEF CMP_AI              = 1 << CMP_AI_BIT          ;; It decides where to go by itself
DEF CMP_ALIVE           = 1 << CMP_ALIVE_BIT

;; Directions: the value of E_DIR, the index of the tables DirVelocity and DirImage
DEF DIR_DOWN  = 0
DEF DIR_UP    = 1
DEF DIR_LEFT  = 2
DEF DIR_RIGHT = 3
DEF DIR_MASK  = %00000011       ;; the 2 bits of a direction

ENDC
```

hardware.inc: registros, OAM, sprites y botones

```
;;=============================================================
;; Challenge 6 · Hardware: registros, pantalla, OAM y botones
;; ❌ No lo modifiques
;;=============================================================
IF !DEF(HARDWARE_INC)
DEF HARDWARE_INC = 1
;;-------------------------------------------------------------
;; Registers
;;-------------------------------------------------------------
DEF rP1    = $FF00              ;; Joypad
DEF rDIV   = $FF04              ;; Divider (changes all the time)
DEF rLCDC  = $FF40              ;; LCD control. Bit 7: LCD on/off
DEF rSCY   = $FF42              ;; Scroll Y: pixels the background moves up
DEF rSCX   = $FF43              ;; Scroll X: pixels the background moves left
DEF rLY    = $FF44              ;; Line being drawn (144-153: VBLANK)
DEF rDMA   = $FF46              ;; DMA: writing a page here copies 160 bytes to OAM
DEF rBGP   = $FF47              ;; Background palette
DEF rOBP0  = $FF48              ;; Sprite palette 0
DEF rOBP1  = $FF49              ;; Sprite palette 1 (sprites with ATTR bit 4)
DEF rWY    = $FF4A              ;; Window: first line where it is drawn
```

```
DEF rWX    = $FF4B              ;; Window: its column + 7

;; Sound. The registers of channel 4 (noise) are the ones the sound effects use
DEF rAUD4LEN  = $FF20           ;; Channel 4: length
DEF rAUD4ENV  = $FF21           ;; Channel 4: volume and how it fades (envelope)
DEF rAUD4POLY = $FF22           ;; Channel 4: noise pitch and type
DEF rAUD4GO   = $FF23           ;; Channel 4: bit 7 starts it, bit 6 uses the length
DEF rAUDVOL   = $FF24           ;; Master volume (bits 6-4 left, bits 2-0 right)
DEF rAUDTERM  = $FF25           ;; Which channels go to which speaker (1 bit each)
DEF rAUDENA   = $FF26           ;; Bit 7: sound on/off (off: other writes are lost)
DEF AUDENA_ON   = %10000000         ;; sound on
DEF AUDTERM_ALL = %11111111         ;; the 4 channels on both speakers
DEF AUDVOL_MAX  = %01110111         ;; maximum volume on both speakers

DEF VBLANK_LINE   = 144
DEF SCREEN_WIDTH  = 160
DEF SCREEN_HEIGHT = 144
DEF LCDC_ON_BIT   = 7
DEF LCDC_VALUE    = %11110111   ;; LCD on · window map $9C00 · window on ·
                                ;; tiles at $8000 · background map $9800 ·
                                ;; sprites 8x16 · sprites on · background on
DEF PALETTE       = %11100100   ;; colour 0 = white ... colour 3 = black
DEF PALETTE_HURT  = %00011011   ;; the colours reversed (a hurt hero flashes with it)

DEF VRAM_TILES    = $8000
DEF TILEMAP       = $9800       ;; the background: 32 x 32 tiles = 256 x 256 pixels
DEF WINDOW_MAP    = $9C00       ;; the window (the HUD): it does not scroll
DEF TILEMAP_WIDTH = 32
DEF OAM_START     = $FE00
DEF OAM_SIZE      = 160         ;; 40 sprites x 4 bytes
DEF HRAM_START    = $FF80
DEF WRAM_END      = $E000

;;-------------------------------------------------------------
;; Sprites: ATTR bits (the 4th byte of a sprite in the OAM)
;;-------------------------------------------------------------
DEF ATTR_PALETTE_BIT = 4        ;; 0 = palette 0 (rOBP0), 1 = palette 1 (rOBP1)
DEF ATTR_XFLIP_BIT   = 5        ;; 1 = mirrored horizontally
DEF ATTR_YFLIP_BIT   = 6        ;; 1 = mirrored vertically
DEF ATTR_PALETTE     = 1 << ATTR_PALETTE_BIT
DEF ATTR_XFLIP       = 1 << ATTR_XFLIP_BIT
DEF ATTR_YFLIP       = 1 << ATTR_YFLIP_BIT

;; Sprites are 8x16: each one shows two tiles, tile N above tile N + 1.
;; An entity (16x16 pixels) is drawn with 2 sprites, side by side.
DEF SPRITES_PER_ENTITY = 2
DEF SPRITE_WIDTH       = 8      ;; pixels
DEF TILES_PER_SPRITE   = 2      ;; an 8x16 sprite shows 2 tiles
;; The position of a sprite in the OAM is the position on the screen plus this, so that
;; a sprite can be partly outside the screen (Y = 0 or X = 0: completely hidden)
DEF OAM_Y_OFFSET       = 16
DEF OAM_X_OFFSET       = 8
;; How the 40 OAM SPRITE slots are used:
;;   slots  0-23  the entities (2 sprites each): DMA copies shadow_oam every frame
;;   slots 24-39  unused (hidden)
;;-------------------------------------------------------------
;; Buttons (input_held / input_pressed). 1 = pressed
```

```
;;-----------------------------------------------------------------
DEF BTN_RIGHT_BIT  = 0
DEF BTN_LEFT_BIT   = 1
DEF BTN_UP_BIT     = 2
DEF BTN_DOWN_BIT   = 3
DEF BTN_A_BIT      = 4
DEF BTN_B_BIT      = 5
DEF BTN_SELECT_BIT = 6
DEF BTN_START_BIT  = 7
DEF BTN_A          = 1 << BTN_A_BIT
DEF BTN_START      = 1 << BTN_START_BIT

ENDC
```

gfx.inc: los IDs de los tiles

```
;;=================================================================
;; Tile IDs of the graphics (all in the $8000 tile block)
;;=================================================================
IF !DEF(GFX_INC)
DEF GFX_INC = 1

DEF NUM_TILES = 148
;; The first tiles are the font: tile = ASCII code - ASCII_TILE_OFFSET
;; (space, punctuation, digits, capitals: ASCII $20-$5F  ->  tiles 0-63)
DEF ASCII_TILE_OFFSET = $20
DEF ASCII_TILE_COUNT  = 64
DEF TILE_BLANK        = 0
DEF TILE_DIGIT_0      = 16
;; Level tiles. From TILE_FIRST_SOLID on, they are SOLID: nothing walks through them
DEF TILE_GRASS_A      = 64
DEF TILE_GRASS_B      = 65
DEF TILE_FLOWER       = 66
DEF TILE_PATH         = 67
DEF TILE_FLOOR        = 68
;; Tiles of the HUD (they never appear in a level map)
DEF TILE_HEART_FULL   = 69
DEF TILE_HEART_EMPTY  = 70
DEF TILE_ICON_ENEMY   = 71
;; Solid tiles of a level
DEF TILE_FIRST_SOLID  = 72
DEF TILE_TREE         = 72
DEF TILE_ROCK         = 73
DEF TILE_WATER        = 74
DEF TILE_WALL         = 75
DEF TILE_BRICK        = 76
;; Entity images: 16x16 pixels = 4 tiles each.
;; A sheet: 3 rows (down, up, side) of 2 images (walking steps): image = row * 2 + step
DEF TILE_PLAYER       = 80
DEF TILE_ENEMY        = 104
DEF TILE_SWORD        = 128
DEF TILE_PORTAL       = 136
DEF TILE_POOF         = 144

DEF TILES_PER_IMAGE = 4        ;; an entity image: 2 sprites of 8x16 = 4 tiles
DEF SHEET_ROW_TILES = 2 * TILES_PER_IMAGE   ;; a row of a sheet: 2 images
ENDC
```

man_map.inc: el mapa

```
;;=================================================================
;; Challenge 6 · Gestor del mapa: constantes
;; ❌ No lo modifiques
;;=================================================================
IF !DEF(MAN_MAP_INC)
DEF MAN_MAP_INC = 1

;; The map of a level is a table of tiles, in ROM: MAP_HEIGHT rows of MAP_WIDTH tiles.
;; It is exactly the size of the background of the console (32 x 32 tiles), so
;; the same bytes are copied to the screen and used to know what is solid.
DEF MAP_WIDTH   = 32                ;; tiles
DEF MAP_HEIGHT  = 32
DEF MAP_SIZE    = MAP_WIDTH * MAP_HEIGHT
DEF TILE_SIZE   = 8                 ;; a tile is 8 x 8 pixels
DEF MAP_PIXELS  = MAP_WIDTH * TILE_SIZE ;; 256: the world is 256 x 256 pixels, so a
                                    ;; position (Y or X) fits in ONE byte

ENDC
```

scn_game.inc: constantes del juego

```
;;=================================================================
;; Challenge 6 · Escena del juego: constantes
;; ❌ No lo modifiques
;;=================================================================
IF !DEF(GAME_INC)
DEF GAME_INC = 1

;; KIND (one bit per kind, so a mask can select several)
DEF KIND_PLAYER = %0001
DEF KIND_SWORD  = %0010
DEF KIND_ENEMY  = %0100
DEF KIND_PORTAL = %1000

;; The hero is always the first entity created after man_entity_init
DEF PLAYER_ENTITY EQUS "entity_array"
;;-----------------------------------------------------------------
;; Game rules (all positions are WORLD pixels: 0-255, the map is 256 x 256)
;;-----------------------------------------------------------------
DEF HEARTS_MAX    = 3       ;; hearts of the hero at the start of a game
DEF NUM_LEVELS    = 3       ;; levels in LevelTable (assets/levels/levels.asm)
DEF WALK_SPEED    = 1       ;; pixels per frame of anything that walks
DEF HURT_FRAMES   = 60      ;; frames the hero is immune after a hit (and flashes)
DEF SWORD_FRAMES  = 10      ;; frames the sword stays out
DEF POOF_FRAMES   = 12      ;; frames the cloud of a defeated enemy stays
DEF AI_THINK_MIN  = 24      ;; an enemy chooses a new way in 24 + (0..31) frames
DEF AI_THINK_MASK = %00011111
DEF AI_STILL_BIT  = 2       ;; bit 2 of a random number: stand still (half the time)
DEF ANIM_STEP_MASK = %00001000   ;; bit 3 of TICK: the step changes every 8 frames
;; Two entities touch if they are closer than this (|dY| < HIT_H and |dX| < HIT_W).
;; Their images are 16 x 16: a smaller distance makes touching a bit more forgiving.
DEF SWORD_REACH   = 12      ;; the sword appears this far from the hero, where it faces
DEF HURT_FLASH_MASK = %00000100  ;; bit 2 of hurt_timer: palette changes every 4 frames
DEF HIT_H         = 12
DEF HIT_W         = 12
;; The part of an entity that the map stops: its FEET, a box of 8 x 8 pixels (the
```

```
;; size of a tile, so any tile it touches has one of its 4 corners inside). Offsets
;; from the top left corner of the entity: the box goes from FEET_LEFT to FEET_RIGHT,
;; and from FEET_TOP to FEET_BOTTOM
DEF FEET_LEFT     = 4
DEF FEET_RIGHT    = 11
DEF FEET_TOP      = 8
DEF FEET_BOTTOM   = 15

ENDC
```

hud.inc: el HUD en la window

```
;;=============================================================
;; Challenge 6 · HUD: corazones, nivel y enemigos
;; ❌ No lo modifiques
;;=============================================================
IF !DEF(HUD_INC)
DEF HUD_INC = 1

INCLUDE "engine/hardware.inc"

;; The HUD lives in the WINDOW: a layer that is drawn over the background and does
;; NOT scroll. Here it is a strip of one row of tiles (8 pixels) at the bottom of
;; the screen.
DEF HUD_WINDOW_X  = 7                     ;; rWX: window at column 0 (WX = column + 7)
DEF HUD_WINDOW_Y  = SCREEN_HEIGHT - 8     ;; rWY: the last row of tiles of the screen
DEF HUD_HIDDEN_Y  = SCREEN_HEIGHT         ;; rWY below the screen: no window

DEF HUD_HEARTS_CELL = WINDOW_MAP + 1      ;; the hearts: 3 cells
DEF HUD_LEVEL_CELL  = WINDOW_MAP + 6      ;; the word NIVEL and its number
DEF HUD_ENEMY_CELL  = WINDOW_MAP + 15     ;; an enemy icon, an X and how many are left

ENDC
```

levels.inc: el formato de un nivel

```
;;=============================================================
;; Challenge 6 · Los niveles del juego: estructura
;; ❌ No lo modifiques
;;=============================================================
IF !DEF(LEVELS_INC)
DEF LEVELS_INC = 1

;; The structure of a level (the offsets of its fields).
;; The levels are in assets/levels/levels.asm
RSRESET
DEF LVL_MAP        RW 1            ;; 0  the address of its map (MAP_SIZE tiles)
DEF LVL_SONG       RW 1            ;; 2  the address of its song (sound_play_song)
DEF LVL_PLAYER_TX  RB 1            ;; 4  where the hero starts: column, row (tiles)
DEF LVL_PLAYER_TY  RB 1            ;; 5
DEF LVL_EXIT_TX    RB 1            ;; 6  where the portal appears
DEF LVL_EXIT_TY    RB 1            ;; 7
DEF LVL_ENEMIES    RB 1            ;; 8  how many enemies, then each one's TX, TY
DEF LVL_ENEMY_LIST RB 0            ;; 9

ENDC
```

Datos

Dato	Bytes	Qué guarda	Lo escribe
`entity_array`	192	Las 12 entidades de 16 bytes. Alineado (`$xx00`)	`man_entity`
`shadow_oam`	160	Copia de la OAM, en WRAM. Alineada (`$xx00`)	pre-render
`shadow_cursor`	1	Dónde va el próximo sprite en `shadow_oam` (su byte bajo)	`sys_prerender`
`cam_x` , `cam_y`	1 + 1	La cámara. **Privados**: se piden con `sys_camera_get`	`sys_camera`
`map_current`	2	Dirección del mapa del nivel actual	`man_map`
`level_current` , `enemies_left`	1 + 1	Nivel que se juega y enemigos que quedan	`scn_game_rules`
`hud_dirty`	1	1: el HUD hay que redibujarlo	`hud`
`rng_state`	2	Estado del generador de números aleatorios	`rng`

Rutinas ya resueltas

⚠ **Estas rutinas ya funcionan. Úsalas tal cual: importa su contrato, no su código.**

Contrato: 📥 entrada · ⬅ salida · ❌ registros que modifica

Rutina	Qué hace	Contrato
`memcpy8`	Copia `B` bytes	📥 `HL` origen, `DE` destino, `B` bytes (1-255) ⬅ `HL` y `DE` han avanzado `B` bytes ❌ `A` , `B` , `DE` , `HL`
`memset8`	Rellena `B` bytes	📥 `HL` destino, `A` valor, `B` bytes (1-255) ❌ `B` , `HL`
`dma_run`	Lanza el DMA: copia la `shadow_oam` a la OAM (ejecuta `hDMARoutine`)	❌ `A`
`sys_camera_get`	Devuelve la cámara	⬅ `B` = cámara Y, `C` = cámara X ❌ `A` , `BC`
`sys_camera_update`	Pone la cámara sobre una entidad	📥 `DE` = entidad ❌ `A` , `BC` , `HL`
`map_load`	Dibuja un mapa en el background (pantalla apagada)	📥 `HL` = mapa ❌ `A` , `BC` , `DE` , `HL`

Rutina	Qué hace	Contrato
`sys_physics_blocked`	¿Hay una esquina de los pies en un tile sólido? (usa `map_is_solid_at`, **el Ejercicio 3**)	📥 `DE` = entidad ⬅ `NZ` si sí, `Z` si no ❌ `A`, `BC`, `HL`
`sys_physics_walk`	La entidad mira hacia una dirección y camina hacia ella	📥 `DE` = entidad, `B` = dirección ❌ `A`, `BC`, `HL`
`sys_physics_stop`	La entidad se queda quieta (sigue mirando igual)	📥 `DE` = entidad ❌ `A`, `HL`
`rng_next`	Un número aleatorio	⬅ `A` = número (0-255) ❌ `A`, `B`, `HL`. **NO** `DE`, **NO** `C`
`level_get`	Los datos del nivel que se juega	⬅ `HL` = dirección de los datos del nivel ❌ `A`, `DE`, `HL`
`game_spawn_at`	Crea una entidad desde una plantilla en un tile	📥 `HL` = plantilla, `C` = columna, `B` = fila ⬅ `Z` si se creó, `NZ` si no había hueco ❌ `A`, `BC`, `HL`. **NO** `DE`
`world_to_oam`	Del mundo a la OAM (**es el Ejercicio 8**)	📥 `D` = Y, `E` = X del mundo ⬅ `B` = Y, `C` = X en la OAM ❌ `A`, `BC`. **NO** `DE`, **NO** `HL`
`sys_ai_choose`	Decide qué hace la entidad con un número aleatorio (**es el Ejercicio 7**)	📥 `DE` = entidad, `A` = número ❌ `A`, `BC`, `HL`

⚠ Cada ejercicio se resuelve aislado
Si un ejercicio usa una rutina de otro ejercicio (por ejemplo, el 9 usa `world_to_oam` y el 6 usa `sys_ai_choose`), asume que existe y cumple **exactamente** su contrato.

🔥 `entity_array` y `shadow_oam` están alineados
Asume que `entity_array` y `shadow_oam` se almacenan ***alineados*** (`ALIGN[8]`): su dirección es `$xx00`. La parte alta (`xx`) no cambia para ninguno de sus elementos. Por eso, para pasar de un campo a otro de una entidad basta con cambiar el byte bajo (`inc l`).

Ejercicio 1

`DMARoutine` y `dma_init`: Encuentra y corrige los errores

Bloque 1. Copiar los 160 bytes de la `shadow_oam` a la OAM con el hardware (DMA). La rutina que lo lanza se ejecuta **desde HRAM**, y `dma_init` la copia allí una sola vez.

`DMARoutine` es el código que ejecutará la CPU desde HRAM:

1. Escribe en `rDMA` la página de `shadow_oam` (`HIGH(shadow_oam)`). La copia empieza ahí.
2. Espera a que acabe con un bucle de `DMA_WAIT_LOOPS` vueltas (`dec` y `jr nz`).
3. Vuelve con `ret` .

`dma_init` copia `DMARoutine` (que está en la ROM) a `hDMARoutine` (en HRAM) con `memcpy8` . El tamaño lo calcula el ensamblador: `DMARoutineEnd - DMARoutine` .

El código completo viene **con errores de todo tipo**. Encuéntralos y arréglalos.

Pasos (intenta antes sin mirarlos)

1. Escribe a un lado lo que debería hacer cada línea de `DMARoutine` , según la descripción de arriba.
2. Compara con el código, línea a línea. Si hay un bucle, cuenta las vueltas que da.
3. Haz lo mismo con `dma_init` , con el contrato de `memcpy8` delante (qué va en `HL` , en `DE` y en `B`).
4. Ensambla: algún error lo encontrará el ensamblador.

Consejos

- `dma_run` ejecuta `hDMARoutine` con un `jp` : ¿cómo vuelve a quien la llamó?
- Todo lo que necesitas para encontrar los errores está en la descripción y en el contrato de `memcpy8` : son errores de bucles, retornos, registros y nombres.

Si funciona

- Los sprites aparecen en la pantalla. Si no se ve ninguno, prueba también los ejercicios 8 y 9: el dibujo usa los tres.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1,** `dma_init` **.** La HRAM (`hDMARoutine`) está llena de EE. Tras `dma_init` , ¿qué bytes hay y cuántos?
- **P2, la copia.** `shadow_oam` = 00 01 02 … 9F (160 bytes) y la OAM, llena de FF. Se ejecuta `dma_init` y después `hDMARoutine` . ¿Cómo queda la OAM al volver?

DMARoutine y dma_init (con errores)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; This is the code that will run from HRAM (see dma_init). Its addresses are absolute
;; (ldh [rDMA]...) and it uses no jumps to the outside, so it works from any place.
DMARoutine:
   ld   a, HIGH(shadow_oam)     ;; \ rDMA = the page of the shadow OAM:
   ldh  [rDMA], a               ;; / the copy starts now
   ld   a, DMA_WAIT_LOOP        ;; \ Wait for the copy to end,
.wait                           ;; | doing nothing (only HRAM can be read):
   dec  a                       ;; | 40 loops x 4 cycles
   jr   z, .wait                ;; /
DMARoutineEnd:

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies the DMA routine to HRAM. It is done once, when the console starts.
;; ❌ Modifies A, B, DE, HL
dma_init::
   ld   hl, hDMARoutine         ;; \ HL = the routine in the ROM
   ld   de, DMARoutine          ;; | DE = its place in HRAM
   ld   b, DMARoutineEnd - DMARoutine ;; | B = its size
   jp   memcpy8                 ;; / Copy it
```

Ejercicio 2

Implementa `camera_axis` y `sys_render_scroll`

Bloque 2. La cámara es la esquina de arriba a la izquierda de lo que se ve del mundo. `camera_axis` la calcula para **un eje** y `sys_render_scroll` la aplica al background.

`camera_axis` : la cámara es `posición - offset` , sin bajar de 0 y sin pasar del máximo.

1. `A` = posición de la entidad en ese eje, `B` = offset (dónde queda la entidad en la pantalla), `C` = máximo de la cámara.
2. Calcula `A - B` . Si el resultado baja de 0 (acarreo), devuelve 0.
3. Si el resultado es menor que `C` , devuélvelo. Si no, devuelve `C` .

`sys_render_scroll` : mueve el background a la posición de la cámara.

1. Pide la cámara con `sys_camera_get` .
2. Escribe la cámara X en `rSCX` y la cámara Y en `rSCY` .

Pasos (intenta antes sin mirarlos)

1. `camera_axis` : `sub b` . Si hay acarreo, `xor a` y `ret` .
2. `cp c` : si hay acarreo (`A < C`), `ret` . Si no, `ld a, c` y `ret` .
3. `sys_render_scroll` : `call sys_camera_get` .
4. Pasa por `A` para escribir cada registro: `ld a, c` y `ldh [rSCX], a` ; `ld a, b` y `ldh [rSCY], a` .

Consejos

- `cp c` calcula `A - C` sin guardar el resultado, y deja el acarreo a 1 si `A < C` .
- `sys_camera_get` te da `B` = Y y `C` = X. Fíjate bien en cuál va a cada registro.

Si funciona

- El mundo se desplaza cuando caminas, y se queda quieto en los bordes del mundo.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, caso normal.** `camera_axis` con `A` = 120, `B` = 72, `C` = 96. `sys_render_scroll` con la cámara en X = 48, Y = 30.
- **P2, los límites.** `camera_axis` con `A` = 30, `B` = 72, `C` = 96, y de nuevo con `A` = 250. `sys_render_scroll` con la cámara en X = 0, Y = 0.

camera_axis y sys_render_scroll

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; The camera for ONE axis: position - offset, not below 0 and not above the maximum.
;; 📥 A = position of the entity in that axis, B = offset, C = maximum of the camera
;; 🡸 A = the camera in that axis
;; ❌ Modifies A
camera_axis::
   ;; ◉◉YOUR CODE◉◉

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves the background to the position of the camera: the scroll. The registers
;; rSCX and rSCY say which pixel of the background is at the top left of the screen.
;; ⚠️ Changing them in the middle of a frame tears the picture: do it in VBLANK
;; (or with the screen off). Use sys_camera_get
;; ❌ Modifies A, BC
sys_render_scroll::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 3

Completa `map_tile_address` e implementa `map_is_solid_at`

Bloque 3. El mapa de un nivel es una tabla de 32 filas de 32 tiles (1.024 bytes) y cada tile mide 8 × 8 píxeles. `map_tile_address` dice dónde está el tile de un punto y `map_is_solid_at`, si ese tile es sólido.

`map_tile_address`: dirección del tile que cubre un punto del mundo.

- 📥 `D` = Y y `E` = X del punto, en píxeles.
- ⬅ `HL` = dirección de su tile en el mapa: `map_current + fila × 32 + columna`, con `fila = Y / 8` y `columna = X / 8`.

El código dado suma `map_current` a `HL` y vuelve. Tú calculas antes, en `HL`, el offset `fila × 32 + columna`:

1. `fila × 32`: la Y sin sus 3 bits bajos es `fila × 8`; multiplícala por 4.
2. `columna`: la X dividida entre 8.
3. Suma la columna a `fila × 32`.

`map_is_solid_at`: ¿está un punto del mundo en un tile sólido? Son sólidos los tiles de `TILE_FIRST_SOLID` en adelante.

- 📥 `D` = Y y `E` = X del punto, en píxeles.
- ⬅ `NZ` si el punto está en un tile sólido y `Z` si está libre.
- Modifica `A` y `HL`, pero **no** `DE`.

1. Pide a `map_tile_address` la dirección del tile del punto.
2. Lee el tile. Si es menor que `TILE_FIRST_SOLID`, está libre: devuelve `Z`.
3. Si no, es sólido: devuelve `NZ`.

Pasos (intenta antes sin mirarlos)

1. Haz a mano la cuenta de un punto (por ejemplo, `Y` = 216, `X` = 32): fila, columna y offset.
2. `ld a, d` y `and %11111000`: `A` = `fila × 8`. Pásalo a `L` y pon `H` a 0.
3. Dos `add hl, hl` y `HL` = `fila × 32`.
4. `ld a, e` y tres `srl a`: `A` = columna. Súmala a `L` (`add l`, `ld l, a`).
5. `map_is_solid_at`: `call map_tile_address` y `ld a, [hl]`.
6. `cp TILE_FIRST_SOLID`: si hay acarreo (menor), está libre. Si no, es sólido.

Consejos

- `fila × 32` tiene sus 5 bits bajos a 0 y la columna es menor que 32: sumarlas nunca da acarreo, solo cambia `L`.
- Si no sabes por dónde empezar, divide entre 8 con tres desplazamientos de bits a la derecha (`srl a`) y multiplica por 32 con `add hl, hl`.

- Para devolver `Z` basta `xor a`; para devolver `NZ`, `or 1`. Cuidado: `cp` con un árbol (`TILE_TREE` = `TILE_FIRST_SOLID`) deja `Z` a 1.

Si funciona

- El héroe no atraviesa los árboles, las rocas ni el agua. La física hace esta pregunta cuatro veces en cada movimiento.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, un punto libre.** Nivel 1. `D` = 216, `E` = 32: es el tile donde empieza el héroe, un camino. `map_tile_address` y, después, `map_is_solid_at` con el mismo punto.
- **P2, un punto sólido.** Nivel 1. `D` = 255, `E` = 255: el último tile del mapa, un árbol. Igual que P1.

map_tile_address y map_is_solid_at

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Where is the tile that covers a point of the world?
;; The map is a table of rows of MAP_WIDTH (32) tiles, and a tile is 8 x 8 pixels:
;;      tile row    = Y / 8        tile column = X / 8
;;      address     = map + row * 32 + column
;; 📥 D = Y, E = X (pixels of the world)
;; 🡄 HL = address of its tile in the map
;; ❌ Modifies A, HL
map_tile_address::
   ;; ◉◉YOUR CODE◉◉

   ld   a, [map_current]        ;; \ HL += map_current: the address in the map
   add  l                       ;; |
   ld   l, a                    ;; |
   ld   a, [map_current + 1]    ;; |
   adc  h                       ;; |
   ld   h, a                    ;; /

   ret

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Is the point solid? A tile is solid if it is TILE_FIRST_SOLID or higher:
;; the tiles are ordered in the tile block so that one comparison decides it.
;; 📥 D = Y, E = X (pixels of the world)
;; 🡄 NZ, the point is in a solid tile
;;    Z,  it is free
;; ❌ Modifies A, HL. Does NOT modify DE
;; ⚠️ Use map_tile_address
map_is_solid_at::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 4

Completa `hud_show` y `hud_draw_hearts`

Bloque 4. El HUD vive en la **window**, una capa que se dibuja sobre el background y no hace scroll.

`hud_show` : coloca la window en la última fila de tiles de la pantalla.

1. Escribe `HUD_WINDOW_X` en `rWX` .
2. Escribe `HUD_WINDOW_Y` en `rWY` .

`hud_draw_hearts` : dibuja los corazones. `C` = corazones del héroe.

1. Escribe `HEARTS_MAX` celdas seguidas, desde `HUD_HEARTS_CELL` (ya cargada en `HL`).
2. Las primeras `C` llevan `TILE_HEART_FULL` y el resto, `TILE_HEART_EMPTY` .

Pasos (intenta antes sin mirarlos)

1. `hud_show` : `ld a, HUD_WINDOW_X` y `ldh [rWX], a` ; lo mismo con `HUD_WINDOW_Y` y `rWY` .
2. `hud_draw_hearts` : en cada vuelta, ¿quedan corazones llenos (`C` distinto de 0)? Elige el tile.
3. Si era un corazón lleno, resta 1 a `C` .
4. `ld [hl+], a` escribe la celda y avanza. `dec b` y repite mientras `B` no sea 0. `ret` .

Consejos

- `ld a, tile` no cambia los flags: puedes comprobar `C` con `ld a, c` / `or a` y cargar el tile después.
- `HL` ya apunta a la primera celda y `B` ya tiene el número de corazones por dibujar.

Si funciona

- Aparece la fila de corazones abajo y no se mueve cuando haces scroll.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, dos corazones.** `rWX` y `rWY` a 0. `hud_show` , y `hud_draw_hearts` con `C` = 2.
- **P2, sin corazones.** `hud_show` , y `hud_draw_hearts` con `C` = 0.

hud_show y hud_draw_hearts

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Shows the window at the bottom of the screen.
;; ⚠️ The screen must be OFF (or be in VBLANK)
;; ❌ Modifies A
hud_show::
   ;; ◉◉YOUR CODE◉◉

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Draws the hearts of the HUD: HEARTS_MAX cells, the first C full and the rest empty.
;; 📥 C = hearts the hero has
;; ❌ Modifies A, BC, HL
hud_draw_hearts::
   ld   b, HEARTS_MAX           ;; B = hearts to draw
   ld   hl, HUD_HEARTS_CELL     ;; HL = the first cell of the hearts

   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 5

Implementa `game_enemies_spawn`

Bloque 5. Crea a los enemigos de un nivel a partir de sus datos.

📥 `HL` = dirección del campo `LVL_ENEMIES` del nivel. Ahí hay un byte con **cuántos enemigos** son, seguido de **su posición**: dos bytes por enemigo (columna y fila, en tiles).

1. Guarda el número de enemigos en `enemies_left`.
2. Si no hay ninguno, termina.
3. Para cada enemigo, crea una entidad desde `EnemyTemplate` en su columna y su fila, con `game_spawn_at`.

Pasos (intenta antes sin mirarlos)

1. `ld a, [hl+]` lee el número. Guárdalo en `enemies_left`. `HL` ya apunta a la primera posición.
2. Pasa `HL` a `DE` (`game_spawn_at` no modifica `DE`, pero sí `HL`).
3. Si el número es 0, `ret`.
4. En cada vuelta: `C` = columna y `B` = fila (leídas de `[DE]`, avanzando `DE`), `HL` = `EnemyTemplate`, `call game_spawn_at`.
5. Repite tantas veces como enemigos. Cuenta con un registro que `game_spawn_at` no modifique, o guárdalo en la pila.

Consejos

- `game_spawn_at` modifica `A`, `BC` y `HL`. El contador de enemigos que te quedan no puede estar en ninguno de ellos mientras llamas.
- `ld a, [de]` / `inc de` lee un byte y avanza.

Si funciona

- Aparecen los enemigos del nivel, cada uno en su tile, y el HUD muestra cuántos quedan.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, 3 enemigos.** Los datos son `3, (2, 3), (20, 4), (9, 30)`. No hay entidades.
- **P2, ningún enemigo.** Los datos son `0`. `enemies_left` vale 99 antes de llamar.

game_enemies_spawn

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Creates the enemies of a level. In the data of the level, a byte with how many
;; they are is followed by the position of each one: its column and row, in tiles.
;; It also remembers how many there are (enemies_left).
;; 📥 HL = &level.ENEMIES
;; ❌ Modifies A, BC, DE, HL
;; ⚠️ Use game_spawn_at, with EnemyTemplate
game_enemies_spawn::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 6

Implementa `sys_ai_one`

Bloque 6. La IA de un enemigo: cada cierto tiempo, elige qué hacer. `DE` = entidad, que tiene `AI`.

1. Resta 1 al `THINK` de la entidad. Si no ha llegado a 0, termina: todavía no toca elegir.
2. Obtén un número aleatorio con `rng_next`.
3. Pon `THINK` = `AI_THINK_MIN` + (el número aleatorio AND `AI_THINK_MASK`).
4. Elige qué hacer con **ese mismo número**: llama a `sys_ai_choose` con `A` = el número y `DE` = la entidad.

Pasos (intenta antes sin mirarlos)

1. `HL` = `&entidad.THINK`: `ld h, d` / `ld a, e` / `add E_THINK` / `ld l, a`.
2. `dec [hl]` y `ret nz`.
3. `rng_next` modifica `A`, `B` y `HL`: guarda `HL` (`push` / `pop`) y el número (por ejemplo, en `C`).
4. `and AI_THINK_MASK`, `add AI_THINK_MIN` y `ld [hl], a`.
5. Recupera el número en `A` y salta a `sys_ai_choose` (su `ret` vuelve a quien te llamó).

Consejos

- `dec [hl]` activa `Z` cuando el valor llega a 0.
- `rng_next` no modifica `DE` ni `C`.

Si funciona

- Los enemigos cambian de dirección o se paran cada 0,4 a 0,9 segundos.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, toca elegir.** La entidad tiene `THINK` = 1. `rng_state` tiene un valor conocido. ¿Cuánto vale `THINK` después y qué hace la entidad?
- **P2, todavía no.** La entidad tiene `THINK` = 5 y se mueve con `VY` = 1. ¿Qué cambia?

sys_ai_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; AI system, for ONE entity: every so often, it chooses what to do next.
;; Its THINK counts the frames left before it chooses. When it reaches 0:
;;  - THINK = AI_THINK_MIN + a random number (0-31): the time until the next choice.
;;  - It chooses what to do with that same random number (sys_ai_choose).
;; ⚠️ Precondition: the entity has CMP_AI.
;; 📥 DE = entity
;; ❌ Modifies A, BC, HL
sys_ai_one::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 7

Implementa `sys_ai_choose`

Bloque 6. Decide qué hace una entidad a partir de un número aleatorio. 📥 `DE` = entidad, `A` = número aleatorio.

1. Si el **bit 7** de `A` vale 1 (la mitad de las veces), la entidad se queda quieta: `sys_physics_stop`.
2. Si no, camina en la dirección que dicen los **bits 6-5** de `A` (un número de 0 a 3, como `DIR_DOWN`, `DIR_UP`, `DIR_LEFT` y `DIR_RIGHT`): `sys_physics_walk` con `B` = esa dirección.

Pasos (intenta antes sin mirarlos)

1. Lleva los bits 7-5 de `A` a las posiciones 2-0 (por ejemplo, con `swap a` y `rrca`).
2. `bit 2, a`: si vale 1, salta a `sys_physics_stop`.
3. `and DIR_MASK` deja la dirección. Pásala a `B`.
4. Salta a `sys_physics_walk`.

Consejos

- `sys_physics_stop` y `sys_physics_walk` terminan con su propio `ret`: puedes **saltar** a ellos con `jp` en vez de llamarlos.
- Hay más de un camino correcto para llegar a esos bits. Si el resultado es el mismo, vale.

Si funciona

- Los enemigos caminan en las 4 direcciones y, a veces, se paran.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, se queda quieta.** `A` = %10100101. La entidad tenía `VY` = 1, `VX` = 0 y `DIR` = `DIR_DOWN`.
- **P2, camina.** `A` = %01110000. La entidad estaba quieta, mirando hacia abajo.

sys_ai_choose

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; The AI decides what an entity does, from a random number: half of the times it
;; stands still, and the other half it walks in one of the 4 directions.
;;  - Bit 7 of the number: 1 = stand still (sys_physics_stop).
;;  - Bits 6-5 of the number: the direction to walk (sys_physics_walk, B = direction).
;; 📥 DE = entity, A = a random number
;; ❌ Modifies A, BC, HL
;; ⚠️ sys_physics_stop and sys_physics_walk end with their own ret
sys_ai_choose::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 8

Implementa `world_to_oam`

Bloque 7. Las entidades guardan su posición en el **mundo**. Para dibujarlas hay que pasarla a coordenadas de la **OAM**: restar la cámara y sumar el offset de la OAM.

📥 `D` = Y del mundo, `E` = X del mundo.

- `B` = `D` − cámara Y + `OAM_Y_OFFSET`
- `C` = `E` − cámara X + `OAM_X_OFFSET`

Todo en 8 bits. No modifica `DE` ni `HL`.

Pasos (intenta antes sin mirarlos)

1. `call sys_camera_get`: te da `B` = cámara Y y `C` = cámara X. Ya no tienes tu entrada en `B` ni en `C`: por eso la entrada está en `D` y `E`.
2. Y: `ld a, d` / `sub b` / `add OAM_Y_OFFSET` / `ld b, a`.
3. X: lo mismo con `E` y `C`.
4. `ret`.

Consejos

- No compruebes si la entidad se ve. Si queda fuera de la pantalla, la resta "da la vuelta" en 8 bits y sale un valor que la consola no dibuja.
- Cada resultado puede sobreescribir a su cámara: cuando has restado, ya no la necesitas.

Si funciona

- Las entidades aparecen donde deben aunque la cámara se mueva.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, dentro de la pantalla.** Cámara X = 50, Y = 20. `D` = 60, `E` = 100.
- **P2, fuera.** Cámara X = 50, Y = 20. `D` = 2, `E` = 3.

world_to_oam

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Changes a position from WORLD coordinates to OAM coordinates:
;;    OAM = world - camera + offset   (offset: OAM_Y_OFFSET for Y, OAM_X_OFFSET for X)
;; All in 8 bits: if the position is outside the screen, the result wraps around to
;; values that the console does not draw (Y >= 160 or X >= 168).
;; 📥 D = world Y, E = world X
;; 🡄 B = OAM Y, C = OAM X
;; ❌ Modifies A, BC. Does NOT modify DE or HL
;; ⚠️ Use sys_camera_get
world_to_oam::
   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 9

Completa `sys_prerender_one`

Bloque 7. Prepara **una** entidad para dibujarla. Su imagen mide 16 × 16: se dibuja con **2 sprites de 8 × 16**, uno al lado del otro, en la `shadow_oam`, a partir de `shadow_cursor`. 📥 `DE` = entidad.

El código dado calcula la posición en la OAM (`B` = Y, `C` = X de la mitad izquierda), descarta las entidades que invaden el HUD, carga `D` = `TILE`, `E` = `ATTR` y `HL` = dónde escribir, y **escribe el sprite 0** (la mitad izquierda). Si la entidad está **volteada** (bit `ATTR_XFLIP` de `ATTR`), las dos mitades **intercambian sus** `X`: el sprite 0 va a la derecha y el sprite 1, a la izquierda. Tú escribes el resto:

1. **Sprite 1**, la mitad derecha: `Y`, `X`, `TILE` + `TILES_PER_SPRITE` y `ATTR`. Su `X` es 8 píxeles más a la derecha (`SPRITE_WIDTH`), salvo que haya volteo.
2. Guarda en `shadow_cursor` el sitio del siguiente sprite.

Pasos (intenta antes sin mirarlos)

1. Mira cómo escribe el sprite 0 su `X`: `C`, o `C` + `SPRITE_WIDTH` si hay volteo. El sprite 1 hace lo contrario.
2. Sprite 1: `Y`, su `X`, su tile (`D` + `TILES_PER_SPRITE`) y `ATTR`.
3. `ld a, l` y `ld [shadow_cursor], a`: `HL` ya está 8 bytes más allá.
4. `ret`.

Consejos

- El sprite 0 se salta el `add SPRITE_WIDTH` con `jr z`. Para el sprite 1, ¿cuál es la condición contraria?
- `bit n, e` no modifica `E` ni `A`: puedes usarlo justo antes de decidir cuál es la X.

Si funciona

- El héroe y los enemigos se ven completos, y al mirar a la izquierda no se rompen por la mitad.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, sin voltear.** Una entidad con `ATTR` = 0, dentro de la pantalla. `shadow_cursor` = 40.
- **P2, volteada.** La misma entidad, con `ATTR` = $20 (volteo en X). `shadow_cursor` = 40.

sys_prerender_one

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares ONE entity to be drawn. Its image is 16 x 16 pixels, drawn with 2
;; sprites of 8 x 16 (each one shows two tiles, one above the other):
;;      sprite 0: the left half, its tile is entity.TILE
;;      sprite 1: the right half, 8 pixels to the right, its tile is entity.TILE + 2
;; If the image is mirrored (ATTR_XFLIP), each half is mirrored and they swap sides.
;; ⚠️ Precondition: the entity has CMP_SPRITE.
;; 📥 INPUT: DE: entity
;; ❌ May modify A, BC, DE, HL
sys_prerender_one::
   ld   h, d                    ;; \ HL = &entity.Y
   ld   a, e                    ;; |
   add  E_Y                     ;; |
   ld   l, a                    ;; /

   ld   a, [hl+]                ;; \ D = entity.Y (HL -> X)
   ld   d, a                    ;; /
   ld   e, [hl]                 ;; E = entity.X
   call world_to_oam            ;; B = OAM Y, C = OAM X of the left half

   ld   a, b                    ;; \ Does its bottom (OAM Y) reach the HUD, or
   cp   HUD_WINDOW_Y + 1        ;; | is it below the screen (the Y wrapped around)?
   ret  nc                      ;; / Yes: not drawn

   inc  l                       ;; \ D = entity.TILE
   ld   d, [hl]                 ;; /
   inc  l                       ;; \ E = entity.ATTR
   ld   e, [hl]                 ;; /

   ld   h, HIGH(shadow_oam)     ;; \ HL = where to write: the shadow OAM + the cursor
   ld   a, [shadow_cursor]      ;; |
   ld   l, a                    ;; /

   ld   a, b                    ;; \ Sprite 0: the left half
   ld   [hl+], a                ;; | Y
   ld   a, c                    ;; | X: normally on the left; mirrored, on the right
   bit  ATTR_XFLIP_BIT, e       ;; |
   jr   z, .left                ;; |
   add  SPRITE_WIDTH            ;; |
.left
   ld   [hl+], a                ;; |
   ld   a, d                    ;; | TILE
   ld   [hl+], a                ;; |
   ld   a, e                    ;; | ATTR
   ld   [hl+], a                ;; /

   ;; ◉◉YOUR CODE◉◉
```

Ejercicio 10

`sys_animation_one` : Encuentra y corrige los errores

Bloque 8. Elige la imagen que muestra una entidad. `DE` = entidad, que tiene `ANIMATION`.

- Su imagen es `TILE` = `SHEET` + la fila de su dirección (`DirImage`) + el paso. `ATTR` lleva el volteo de su dirección, sin tocar el resto de sus bits.
- El paso cambia cada 8 frames: es el bit 3 de `TICK`, que se pasa a 0 o a `TILES_PER_IMAGE`.
- Una entidad con `PHYSICS` solo se anima mientras se mueve (`VY` o `VX` distintos de 0). Si está quieta, `TICK` vuelve a 0 (la primera imagen). Las que no tienen `PHYSICS` (el portal) se animan siempre.

El código completo viene **con errores de todo tipo**. Encuéntralos y arréglalos.

Pasos (intenta antes sin mirarlos)

1. Escribe a un lado la fórmula de la imagen y las dos reglas (¿cuándo se anima?).
2. Recorre el código bloque a bloque y escribe qué hace cada uno.
3. Compara. Prueba con un caso concreto: `DIR_LEFT`, `TICK` = 7.
4. Ensambla: algún error lo encontrará el ensamblador.

Consejos

- Cada entrada de `DirImage` ocupa 2 bytes. Mira cómo se calcula su índice.
- `res` borra un bit de `ATTR`; `or` pone el volteo. ¿Es el bit correcto?

Si funciona

- Los personajes mueven las piernas al caminar, se quedan quietos al parar, y miran hacia donde caminan.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, camina a la izquierda.** Con `PHYSICS`, `VX` = −1, `DIR_LEFT`, `TICK` = 7, `SHEET` = 80, `ATTR` = 0.
- **P2, quieta.** Con `PHYSICS`, `VY` = `VX` = 0, `DIR_UP`, `TICK` = 13, `SHEET` = 80, `ATTR` = $30 (paleta 1 y volteo en X).

sys_animation_one (con errores)

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Animation system, for ONE entity. The image it shows is:
;;      TILE = SHEET + the row of its direction + the image of its step
;; and ATTR gets the flip of its direction. TICK counts the frames it has been walking;
;; its step changes every 8 frames, so it alternates between its 2 images.
;; An entity with PHYSICS only animates while it moves (VY or VX not 0): when it
;; stands still, it shows the first image. The others (a portal) always animate.
;; ⚠️ Precondition: the entity has CMP_ANIMATION.
;; 📥 DE = entity
;; ❌ Modifies A, BC, HL
sys_animation_one::
   ld   a, [de]                 ;; \ A = entity.CMP
   and  CMP_PHYSICS             ;; | Does it have PHYSICS?
   jr   nz, .tick               ;; / No: it always animates

   ld   h, d                    ;; \ A = VY | VX: it is 0 only if it is standing still
   ld   a, e                    ;; | (HL = &entity.VY: same page as the entity,
   add  E_VY                    ;; | the offset added to the low byte)
   ld   l, a                    ;; |
   ld   a, [hl+]                ;; |
   or   [hl]                    ;; /
   jr   nz, .tick               ;; Moving: it animates

   ld   h, d                    ;; \ Standing still: TICK = 0, the first image
   ld   a, e                    ;; | (HL = &entity.TICK)
   add  E_TICK                  ;; |
   ld   l, a                    ;; |
   ld   [hl], 0                 ;; |
   jr   .image                  ;; /

.tick
   ld   h, d                    ;; \ One more frame walking:
   ld   a, e                    ;; | entity.TICK++
   add  E_TICK                  ;; | (HL = &entity.TICK)
   ld   l, a                    ;; |
   inc  [hl]                    ;; /

.image
   ld   a, [hl]                 ;; \ A = TICK
   and  ANIM_STEP_MASK          ;; | Bit 3 of TICK: 0 for 8 frames, 8 for the next 8...
   ld   b, a                    ;; / B = the tile offset of its step (0 or 4)

   ld   h, d                    ;; \ A = entity.DIR
   ld   a, e                    ;; |
   add  E_DIR                   ;; |
   ld   l, a                    ;; |
   ld   a, [hl]                 ;; /
   ld   c, a                    ;; \ BC = DIR * 2: the offset in the table
   ld   a, b                    ;; | A = the offset of the step
   ld   b, 0                    ;; /
   ld   hl, DirImages           ;; \ HL = &DirImage[DIR]
   add  hl, bc                  ;; /

   add  [hl]                    ;; \ A = step offset + tile of the row of its direction
   inc  hl                      ;; | (HL -> the flip)
   ld   b, [hl]                 ;; / B = the flip of its direction
```

```
    ld   c, a                   ;; C = that tile (A is needed to build the addresses)

    ld   h, d                   ;; \ HL = &entity.SHEET
    ld   a, e                   ;; |
    add  E_SHEET                ;; |
    ld   l, a                   ;; /
    ld   a, c                   ;; \ C = the tile of the image:
    add  [hl]                   ;; | that tile + entity.SHEET
    ld   c, a                   ;; /

    ld   h, d                   ;; \ HL = &entity.TILE
    ld   a, e                   ;; |
    add  E_TILE                 ;; |
    ld   l, a                   ;; /
    ld   [hl], c                ;; \ entity.TILE = C
    inc  l                      ;; / (HL -> ATTR: the next field)

    ld   a, [hl]                ;; \ entity.ATTR = its ATTR with the flip of
    res  ATTR_YFLIP_BIT, a      ;; | its direction (the other bits stay)
    or   b                      ;; |
    ld   [hl], a                ;; /

    ret
```

Y después

Explica **por qué** funciona cada ejercicio: qué hace cada bloque y qué pasaría sin él. Eso te permitirá adaptarlo en clase.

> **Reflexión**
> Casi ninguna rutina sabe nada de las demás: solo cumple un contrato. ¿Qué te facilita eso al trabajar en equipo?